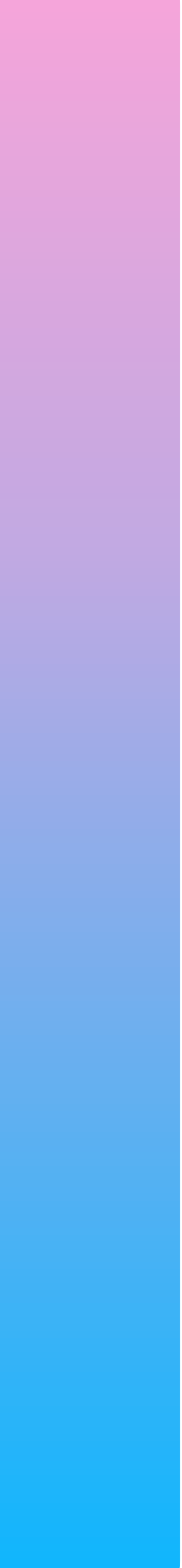


DIVISÃO E CONQUISTA: MERGESORT

Gustavo Amaral Azevedo, Hugo Pereira Pinhata,
Marcus Vinícius Magri, Miguel Totti de Oliveira



RESUMO



- Divisão e Conquista: conceitos fundamentais
- Análise assintótica temporal
- Equações de Recorrência: $T(n) = a T(n/b) + f(n)$
- Teorema Mestre
- Métodos alternativos: Iteração, Árvore, Substituição
- Algoritmo MergeSort
- Encontrar máximo e mínimo simultaneamente
- Contagem de Inversões
- Análise empírica: força bruta vs. divisão e conquista
- Aplicações práticas e comparações de desempenho

INTRODUÇÃO: CONCEITOS FUNDAMENTAIS



Recorrências e Recursividade

Função que **chama a si mesma** até atingir o caso base.

Caso base: condição de parada da recursão.

Equação geral: $T(n) = a \cdot T(n/b) + f(n)$

Divisão e Conquista

Dividir: problema em subproblemas menores.

Conquistar: resolver cada subproblema recursivamente.

Combinar: unir as soluções parciais.

MÉTODOS DE RESOLUÇÃO DE RECORRÊNCIAS



Teorema Mestre: método direto e eficiente para resolver recorrências da forma:

$$T(n) = a T(n/b) + f(n),$$

ideal para casos típicos de divisão e conquista.

Limitação do Teorema Mestre: não se aplica a recorrências do tipo

$$T(n) = a T(n - b) + f(n),$$

onde o subproblema é reduzido por subtração, e não por divisão.

Métodos alternativos para casos mais complexos: Iteração (expansão direta da recorrência), Árvore de Recorrência (visualização dos níveis de custo) e Substituição (hipótese + prova por indução matemática).

MERGESORT: DIVISÃO E CONQUISTA NA PRÁTICA



MergeSort

Funcionamento

Divisão e Conquista

Divide o vetor em sub-vetores recursivamente até tamanho 1 (caso base). Em seguida, **combina** os sub-vetores ordenados (merge), produzindo um **vetor completamente ordenado** a cada nível da recursão.

Equação de Recorrência

$$T(n) = 2T(n/2) + n$$

a = 2 sub-problemas

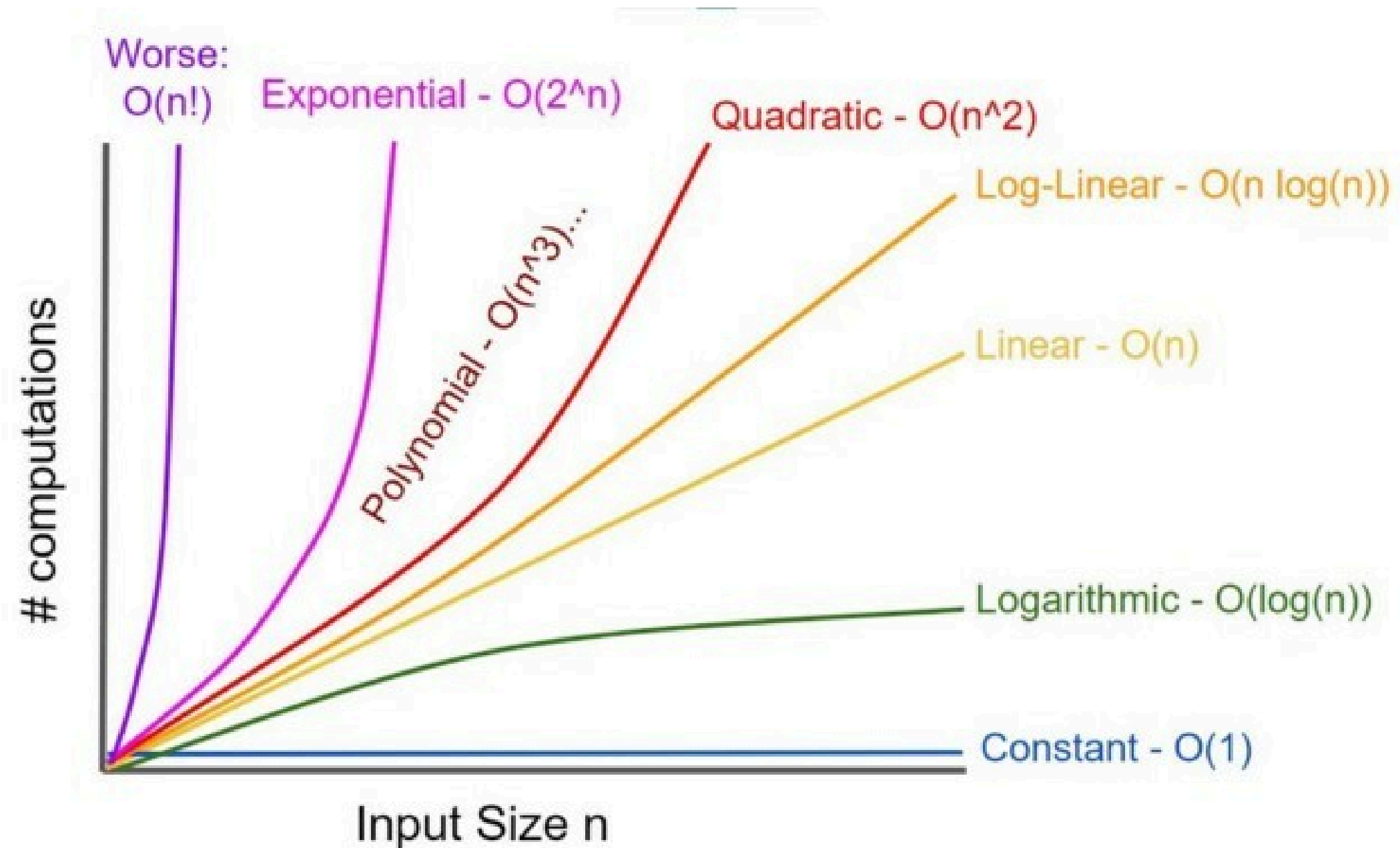
b = diminuição do problema por 2 ($n/2$)

Custo de combinação: $f(n) = n$ (intercalação linear).

Caso base: $T(1) = \Theta(1)$.

COMPLEXIDADE DO MERGESORT

A complexidade do MergeSort é **melhor que quadrática** ($O(n \log n)$), superando algoritmos como Bubble Sort e Insertion Sort ($O(n^2)$).



**ENCONTRAR MÁXIMO E MÍNIMO
COM DIVISÃO E CONQUISTA**



ALGORITMO MÁXIMO E MÍNIMO

```
static (int min, int max) FindMaxMin(int[] arr, int left, int right)
{
    // Caso base 1
    if (left == right)
        return (arr[left], arr[right]);

    // Caso base 2
    if (right == left + 1)
    {
        if (arr[left] < arr[right])
            return (arr[left], arr[right]);
        else
            return (arr[right], arr[left]);
    }

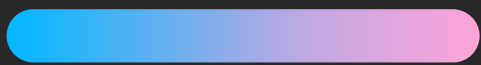
    // Divisão
    int mid = (left + right) / 2;

    // Conquista
    var (minLeft, maxLeft) = FindMaxMin(arr, left, mid);
    var (minRight, maxRight) = FindMaxMin(arr, mid + 1, right);

    // Combinação
    int minGlobal = minLeft < minRight ? minLeft : minRight;
    int maxGlobal = maxLeft > maxRight ? maxLeft : maxRight;

    return (minGlobal, maxGlobal);
}
```

ANÁLISE DE COMPLEXIDADE: MÁXIMO E MÍNIMO



Recorrência e Teorema Mestre

$$T(n) = 2T(n/2) + O(1)$$

$$a=2, b=2, f(n)=O(1)$$

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

$f(n)$ está “**por baixo**” que $n^{\log_b a}$

Resultado e Comparação

Caso 1 do Teorema Mestre:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$$

Ingênua: $2(n-1)$ comparações

Divisão e Conquista: $3n/2 - 2$ comparações
Redução de ~25% no número de comparações

Contagem de Inversões

Definição do Problema

Dado um array A , uma inversão é um par (i, j) tal que $i < j$ e $A[i] > A[j]$. Exemplo: $A = [3, 1, 2]$ possui 2 inversões: $(3,1)$ e $(3,2)$. O objetivo é contar todas as inversões eficientemente.

Restrições e Desafio

$N \leq 5 \times 10^5$ elementos; valores em $[-10^9, +10^9]$. Força bruta $O(n^2)$ é inviável para grandes entradas. Desafio: contar inversões em **$O(n \log n)$** usando **Divisão e Conquista**.

MERGESORT MODIFICADO

```
static long MergeSort(int[] A, int p, int r)
{
    long contador = 0;
    if (p < r)
    {
        int q = (p + r) / 2;
        contador += MergeSort(A, p, q);
        contador += MergeSort(A, q + 1, r);
        contador += Intercala(A, p, q, r);
    }
    return contador;
}
```

```
static long Intercala(int[] A, int p, int q, int r)
{
    int n1 = q - p + 1;
    int n2 = r - q;

    int[] L = new int[n1];
    int[] R = new int[n2];

    for (int i = 0; i < n1; i++) L[i] = A[p + i];
    for (int j = 0; j < n2; j++) R[j] = A[q + 1 + j];

    int iL = 0, iR = 0, k = p;
    long inversoes = 0;

    while (iL < n1 && iR < n2)
    {
        if (L[iL] <= R[iR])
        {
            A[k] = L[iL];
            iL++;
        }
        else
        {
            A[k] = R[iR];
            iR++;
            inversoes += (n1 - iL); // Contabiliza as inversões na intercalação
        }
        k++;
    }

    while (iL < n1) { A[k] = L[iL]; iL++; k++; }
    while (iR < n2) { A[k] = R[iR]; iR++; k++; }

    return inversoes;
}
```

FUNÇÃO INTERCALA

Linha chave: $\text{inversoes} += (n1 - iL);$

Quando $R[iR] < L[iL]$, todos os $(n1 - iL)$ elementos restantes em L são maiores que $R[iR]$, formando inversões.

A soma acumula todas as inversões detectadas durante a intercalação.

Processo: dois ponteiros percorrem L e R; o menor elemento é copiado para o vetor original; ao copiar de R, conta-se $(n1 - iL)$ inversões.

Complexidade: $\Theta(n)$.

Exemplo:



Logo, o numero 1 teria que passar por cima (inversões) de todos os numeros de L, a partir de iL , para chegar na posição correta.

ANÁLISE DE COMPLEXIDADE DA FUNÇÃO INTERCALA

```
static long Intercala(int[] A, int p, int q, int r)
{
    int n1 = q - p + 1;  $\Theta(1)$ 
    int n2 = r - q;       $\Theta(1)$ 

    int[] L = new int[n1];  $\Theta(1)$ 
    int[] R = new int[n2];  $\Theta(1)$ 

    for (int i = 0; i < n1; i++) L[i] = A[p + i];  $\Theta(n)$ 
    for (int j = 0; j < n2; j++) R[j] = A[q + 1 + j];  $\Theta(n)$ 

    int iL = 0, iR = 0, k = p;  $\Theta(1)$ 
    long inversoes = 0;  $\Theta(1)$ 

    while (iL < n1 && iR < n2)
    {
        if (L[iL] <= R[iR])
        {
            A[k] = L[iL];
            iL++;
        }
        else
        {
            A[k] = R[iR];
            iR++;
            inversoes += (n1 - iL); // Contabiliza as inversões na intercalação
        }
        k++;
    }

    while (iL < n1) { A[k] = L[iL]; iL++; k++; }  $\Theta(n)$ 
    while (iR < n2) { A[k] = R[iR]; iR++; k++; }  $\Theta(n)$ 

    return inversoes;  $\Theta(1)$ 
}
```

Soma total das complexidades:

$$T = \Theta(1) + \Theta(n) + \Theta(n) + \Theta(1) + \Theta(n) + \Theta(n) + \Theta(1)$$

$$T = \Theta(3) + \Theta(4n)$$

$$T = \Theta(n)$$

ANÁLISE DE COMPLEXIDADE DO MERGESORT MODIFICADO

```
static long MergeSort(int[] A, int p, int r)
{
    long contador = 0;  $\Theta(1)$ 
    if (p < r)  $\Theta(1)$ 
    {
        int q = (p + r) / 2;  $\Theta(1)$ 
        contador += MergeSort(A, p, q);  $T(n/2)$ 
        contador += MergeSort(A, q + 1, r);  $T(n/2)$ 
        contador += Intercala(A, p, q, r);  $\Theta(n)$ 
    }
    return contador;  $\Theta(1)$ 
}
```

Recorrência do MergeSort com contador:

$$T(n) = \Theta(1) + \Theta(1) + \Theta(1) + T(n/2) + T(n/2) + \Theta(n) + \Theta(1)$$

$$T(n) = 2T(n/2) + \Theta(n) + \Theta(3)$$

$$T(n) = 2T(n/2) + \Theta(n)$$

Utilizando Teorema Mestre:

$$T(n) = \Theta(n \log n)$$

A adição do contador de inversões não altera a complexidade assintótica do MergeSort original, mantendo eficiência ótima para grandes volumes de dados.

ANÁLISE EMPÍRICA

Força Bruta $O(n^2)$

Vetor: 500.000 elementos aleatórios.

Tempo de execução: 9 minutos e 38 segundos

Inversões contadas: 62.548.441.022

Abordagem ingênua com dois laços aninhados. Inviável para grandes volumes de dados em ambientes reais.

MergeSort Modificado $O(n \log n)$

Vetor: 500.000 elementos aleatórios.

Tempo de execução: 0,126 segundos

Inversões contadas: 62.548.441.022

Mesmo resultado, **~99,978% mais rápido**. Divisão e conquista confirma superioridade empírica e teórica sobre a força bruta.

```
Tamanho do vetor: 500000
```

```
--- Teste Força Bruta  $O(n^2)$  ---
```

```
Total de Inversões: 62548441022
```

```
Tempo de Execução: 00:09:38.5031486
```

```
--- Teste MergeSort  $O(n \log n)$  ---
```

```
Total de Inversões: 62548441022
```

```
Tempo de Execução: 00:00:00.1266382
```




OBRIGADO!

P E R G U N T A S ?

